How to merging files in python

Arianty Siahaan
Data Analyst

Prepare your file:
for example
file 1, file 2, file 3, file 4.

A. Merging Daily

```
import pandas as pd
from google.colab import drive

# Mount Google Drive
drive.mount('/content/drive')
```

`import pandas as pd`

This line loads **Pandas**, a Python library used to:

- read Excel or CSV files
- work with tables of data
- analyze and clean data
  pd is just a **short name** so we don't have to type pandas every time.

`from google.colab import drive`

This line lets **Google Colab connect to your Google Drive**.

It is used when:

- your data is stored in Google Drive
- you want to read or save files from Drive in Colab

```
file1 = '/content/drive/MyDrive/Data PT
Check/202510/20251013/592_Respect Jaya Truss Bogor.xlsx'
file2 = '/content/drive/MyDrive/Data PT
Check/202510/20251013/620_ARTAPRIMA CIPTA CATURINDO Depok.xlsx'
```

this one to find the location of your file from your google drive. Each file is in excel format.

```
# Baca file Excel
df1 = pd.read_excel(file1, header=[10, 11])
```

- reads an Excel file into a table (DataFrame)
- uses **row 11 and 12** as the column headers
  (because Python starts counting from 0)

```
# Ambil nama file (tanpa ekstensi) → misalnya jadi: "Shopee"
nama_toko = os.path.splitext(os.path.basename(file1))[0]
```

This line:

- takes the file name from the file path
- removes the `.xlsx` extension
- saves the result as the **store name**

```
df1['Toko'] = nama_toko
```

This:

- adds a new column called **"Toko"**
- fills every row with the store name

```
import datetime
```

This imports a tool to work with **dates**.

```
df1['Tanggal'] = datetime.date(2025,10,13)
```

This:

- creates a new column **"Tanggal"**
- sets the same date for all rows
  (October 13, 2025)

```
df1['Tanggal'] = pd.to_datetime('2025-10-13')  # format ISO atau
bisa juga '05/05/2025'
```

This:

- converts the date into **datetime format**
- makes it easier to filter, sort, or analyze by date

- Combine (stack) two datasets

```
merged_20251013_df = pd.concat([df1, df2], ignore_index=True)
```

This line:

- combines **df1 and df2** into one table
- stacks them **row by row**
- resets the row numbers so they go 0, 1, 2, 3, …

- Remove missing values from a specific column

```
merged_20251013_df = merged_20251013_df.dropna(subset=[('Qty/dus',
'Unnamed: 7_level_1')])
```

This line:

- removes rows that have **empty (missing) values**
- only checks the column **Qty/dus**
- if Qty/dus is empty → that row is deleted

- Set the output file path

```
# Simpan hasil gabungan ke file baru
```

```
output_path = '/content/drive/MyDrive/Data PT
Check/202510/20251013/merged_20251013.xlsx'
```

This:

- defines where the new Excel file will be saved
- saves it inside **Google Drive**

- Save the merged data to Excel

```
merged_20251013_df.to_excel(output_path)
```

This line:

- exports the combined data
- saves it as an **Excel file**

- Print confirmation message

```
print("File berhasil digabung dan disimpan sebagai
'merged_20251013.xlsx'")
```

This:

- shows a message to confirm the process was successful

B. Merging Monthly

It **combines many daily Excel files into one monthly Excel file** (October 2025).

So instead of checking data day by day, you get **one big dataset for the whole month**

Step-by-step explanation

1. Import libraries

```
import pandas as pd

from google.colab import drive
```

- `pandas` → used to read, combine, and clean Excel data
- `drive` → lets Google Colab access your Google Drive

---

2. Connect Google Drive

```
drive.mount('/content/drive')
```

This allows the code to:

- read Excel files from Google Drive
- save the final merged file back to Drive

---

3. List of daily merged files

```
file_paths = [

    '/content/drive/MyDrive/Data PT
Check/202510/20251013/merged_20251013.xlsx',

    ...

]
```

This is a **list of Excel files**:

- each file = **daily merged sales data**
- dates from **13 Oct to 31 Oct 2025**

---

4. Read all Excel files

```
dfs = [pd.read_excel(path) for path in file_paths]
```

This:

- opens each Excel file
- stores them as DataFrames in a list called `dfs`

So now you have:

- df for 13 Oct
- df for 14 Oct
- df for 15 Oct
- etc.

---

5. Merge all daily data into one

```
merged_202510_df = pd.concat(dfs, ignore_index=True)
```

This:

- **stacks all daily data vertically**
- creates **one big table for October**

`ignore_index=True`:

- resets row numbers so they run neatly from top to bottom

---

6. Remove invalid rows

```
merged_202510_df = merged_202510_df.dropna(subset=['Toko'])
```

This removes rows where:

- the **Toko (Store)** column is empty

---

7. Save monthly result

```
merged_202510_df.to_excel(output_path, index=False)
```

This:

- saves the final **October 2025 merged data** into one Excel file
- removes row index so the file looks clean

- Output:

`merged_202510.xlsx`

C. Merging each month (all file)

1. Comment

```
#BACA FILE DARI DRIVE
```

This is just a **note .**

---

2. Import libraries

```
import pandas as pd
```

```
from google.colab import drive
```

- `pandas` → used to read and process Excel data
- `drive` → allows Google Colab to access Google Drive

---

3. Mount Google Drive

```
drive.mount('/content/drive')
```

This:

- connects your Google Drive to Google Colab
- makes Drive files accessible like a normal folder

You usually run this **once per session**.

---

4. Set file path

```
file_path = '/content/drive/MyDrive/Data PT Check/merged_2025.xlsx'
```

This line:

- stores the **location of the Excel file**
- tells Python **where the file is**

To load the Excel file into a DataFrame, you still need:

```
df = pd.read_excel(file_path)
```

After that:

- `df` will contain all the data from `merged_2025.xlsx`
- you can filter, group, visualize, or build dashboards

1. Import libraries

```
import pandas as pd

import re
```

- `pandas` → to read, merge, and process Excel data
- `re` → to extract numbers (product length) from text

---

2. List of monthly files

```
file_paths = [
    'merged_202505.xlsx',
    'merged_202506.xlsx',
    ...
    'merged_202512.xlsx'
]
```

This is a list of **monthly merged files**, from **May to December 2025**.

---

3. Read all Excel files

```
dfs = [pd.read_excel(path) for path in file_paths]
```

Each Excel file is:

- read into a DataFrame
- stored in a list called `dfs`

---

4. Merge all months into one dataset

```
merged_2025_df = pd.concat(dfs, ignore_index=True)
```

This:

- stacks all monthly data together
- creates **one yearly DataFrame for 2025**
- resets the index automatically

- This is the **yearly merging process**.

---

5. Remove rows with missing store name

```
merged_2025_df = merged_2025_df.dropna(subset=['Toko'])
```

- Rows without a **store name (Toko)** are removed
- This ensures only valid sales records are kept

---

6. Extract product length from Description

```
def extract_length(description):
    match = re.search(r'length\s*:\s*(\d+)', description, re.IGNORECASE)
    if match:
        return int(match.group(1))
    return None
```

This function:

- searches for text like `length: 400`
- extracts the number (`400`)
- returns `None` if no length is found

---

7. Apply the function to all rows

```
merged_2025_df['length'] =
merged_2025_df['Description'].astype(str).apply(extract_length)
```

- Creates a new column called **`length`**
- The length is stored in **centimeters (cm)**

---

8. Calculate price per meter

```
merged_2025_df['Harga_per_meter'] = (
    merged_2025_df['UnitPrice(incPPN)'] / (merged_2025_df['length']
/ 100)
)
```

This:

- converts length from **cm to meters**
- calculates **price per meter**

Example:

- Unit price = 100,000

- Length = 400 cm → 4 meters
- Price per meter = 25,000

---

9. Remove unnecessary columns

```
merged_2025_df = merged_2025_df.drop(
    columns=[col for col in kolom_dihapus if col in merged_2025_df.columns]
)
```

- Removes duplicate, unused, or irrelevant columns
- Makes the dataset **cleaner and easier to analyze**

---

10. Save the final yearly file

```
merged_2025_df.to_excel(output_path, index=False)
```

Final result:
-One clean file: **`merged_2025.xlsx`**
- Contains: